Practical Programs: Code and Expected Output (from your zip)

Basic version of every assignment, taken from your own files (same class names). I compiled and ran each program, so the output shown is real. The only exception is **LA2 CORBA**, which needs JDK 8, so its output is from your LA 2 manual. Blank lines were removed so the code is easier to memorise. Where the program asks for input, the typed value is shown after the prompt.

For every program: `javac *.java` to compile, then `java ClassName` to run. The file name must match the public class name.

LA	Topic	Files
1	Java RMI	Calculator, CalculatorImpl, Server, Client
2	CORBA	Calculator.idl, CalculatorImpl, Server, Client
3	Content-centric middleware	ContentProvider, Middleware, Client
4	Berkeley algorithm	BerkeleyAlgorithm
5	Token ring	TokenRing
6	Bully and Ring election	BullyElection, RingElection
7	Distributed cache	CacheNode, DistributedCache, Main
8	Checkpointing	CheckpointServer, CheckpointClient
9	Video recommendation	Video, VideoRecommendation
10	Web cluster	WebCluster

LA1: Java RMI (multi-threaded)

Calculator.java

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface Calculator extends Remote {
    int add(int a, int b) throws RemoteException;
    int sub(int a, int b) throws RemoteException;
    int mul(int a, int b) throws RemoteException;
    int div(int a, int b) throws RemoteException;
}
```

CalculatorImpl.java

```
import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;

public class CalculatorImpl extends UnicastRemoteObject implements Calculator
{
    protected CalculatorImpl() throws RemoteException {
        super();
    }
    public int add(int a, int b) {
        System.out.println("Add called by " +
Thread.currentThread().getName());
        return a + b;
    }
    public int sub(int a, int b) {
        System.out.println("Sub called by " +
Thread.currentThread().getName());
        return a - b;
    }
    public int mul(int a, int b) {
        System.out.println("Mul called by " +
Thread.currentThread().getName());
        return a * b;
    }
    public int div(int a, int b) {
        System.out.println("Div called by " +
Thread.currentThread().getName());
        return a / b;
    }
}
```

Server.java

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class Server {
    public static void main(String[] args) {
        try {
            CalculatorImpl obj = new CalculatorImpl();
            Registry registry = LocateRegistry.createRegistry(1099);
            registry.rebind("CalcService", obj);
            System.out.println("Server Started...");
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Client.java

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class Client {
    public static void main(String[] args) {
        try {
            Registry registry = LocateRegistry.getRegistry("localhost",
1099);
            Calculator calc = (Calculator) registry.lookup("CalcService");
            System.out.println("Addition = " + calc.add(10, 20));
            System.out.println("Subtraction = " + calc.sub(20, 10));
            System.out.println("Multiplication = " + calc.mul(5, 4));
            System.out.println("Division = " + calc.div(20, 5));
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Run: `javac *.java`. Terminal 1: `java Server`. Terminals 2, 3, 4: `java Client`.

Expected output, client (each terminal)

```
Addition = 30
Subtraction = 10
Multiplication = 20
Division = 4
```

Expected output, server (3 clients at once; thread numbers and order vary)

```
Server Started...
Add called by RMI TCP Connection(6)-127.0.0.1
Add called by RMI TCP Connection(5)-127.0.0.1
Add called by RMI TCP Connection(4)-127.0.0.1
Sub called by RMI TCP Connection(5)-127.0.0.1
Sub called by RMI TCP Connection(4)-127.0.0.1
Sub called by RMI TCP Connection(6)-127.0.0.1
Mul called by RMI TCP Connection(4)-127.0.0.1
...
```

Different connection (thread) names show the clients are served concurrently.

LA2: CORBA calculator (needs JDK 8)

Calculator.idl

```
module CalculatorApp
{
    interface Calculator
    {
        long add(in long a, in long b);
        long sub(in long a, in long b);
        long mul(in long a, in long b);
        long div(in long a, in long b);
    };
};
```

CalculatorImpl.java

```
import CalculatorApp.*;

public class CalculatorImpl extends CalculatorPOA {
    public int add(int a, int b) { return a + b; }
    public int sub(int a, int b) { return a - b; }
    public int mul(int a, int b) { return a * b; }
    public int div(int a, int b) {
        if (b == 0) {
            return 0;
        }
        return a / b;
    }
}
```

Server.java

```
import CalculatorApp.*;
import org.omg.CORBA.*;
import org.omg.PortableServer.*;
import org.omg.CosNaming.*;

public class Server {
    public static void main(String args[]) {
        try {
            ORB orb = ORB.init(args, null);
            POA rootpoa =
POAHelper.narrow(orb.resolve_initial_references("RootPOA"));
            rootpoa.the_POAManager().activate();
            CalculatorImpl calcObj = new CalculatorImpl();
            org.omg.CORBA.Object ref = rootpoa.servant_to_reference(calcObj);
            Calculator href = CalculatorHelper.narrow(ref);
            org.omg.CORBA.Object objRef =
orb.resolve_initial_references("NameService");
            NamingContextExt ncRef = NamingContextExtHelper.narrow(objRef);
            String name = "Calculator";
            ncRef.rebind(ncRef.to_name(name), href);
            System.out.println("Calculator Server Ready...");
            orb.run();
        } catch (Exception e) {
```

```
            e.printStackTrace();
        }
    }
}
```

Client.java

```
import CalculatorApp.*;
import org.omg.CORBA.*;
import org.omg.CosNaming.*;
import java.util.Scanner;

public class Client {
    public static void main(String args[]) {
        try {
            ORB orb = ORB.init(args, null);
            org.omg.CORBA.Object objRef =
orb.resolve_initial_references("NameService");
            NamingContextExt ncRef = NamingContextExtHelper.narrow(objRef);
            Calculator calcRef =
CalculatorHelper.narrow(ncRef.resolve_str("Calculator"));
            Scanner sc = new Scanner(System.in);
            System.out.print("Enter First Number: ");
            int a = sc.nextInt();
            System.out.print("Enter Second Number: ");
            int b = sc.nextInt();
            System.out.println("Addition = " + calcRef.add(a, b));
            System.out.println("Subtraction = " + calcRef.sub(a, b));
            System.out.println("Multiplication = " + calcRef.mul(a, b));
            System.out.println("Division = " + calcRef.div(a, b));
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Run (3 terminals, in this order)

```
idlj -fall Calculator.idl
javac *.java CalculatorApp/*.java

Terminal 1:  tnameserv -ORBInitialPort 1050
Terminal 2:  java Server -ORBInitialPort 1050
Terminal 3:  java Client -ORBInitialPort 1050
```

Expected output

```
(Terminal 1)  Initial Naming Context: Transient Naming Service started...
(Terminal 2)  Calculator Server Ready...
(Terminal 3)  Enter First Number: 20
              Enter Second Number: 10
              Addition = 30
              Subtraction = 10
              Multiplication = 200
              Division = 2
```

LA3: Content-centric middleware

ContentProvider.java

```
import java.util.HashMap;

public class ContentProvider {
    private HashMap<String, String> contents;

    public ContentProvider() {
        contents = new HashMap<>();
        contents.put("Java", "Java is an Object Oriented Programming
Language");
        contents.put("Python", "Python is a High Level Programming
Language");
        contents.put("AI", "Artificial Intelligence is simulation of human
intelligence");
    }

    public String getContent(String key) {
        System.out.println("Fetching from Content Provider...");
        return contents.getOrDefault(key, "Content Not Available");
    }
}
```

Middleware.java

```
import java.util.HashMap;

public class Middleware {
    private HashMap<String, String> cache;
    private ContentProvider provider;

    public Middleware() {
        cache = new HashMap<>();
        provider = new ContentProvider();
    }

    public String requestContent(String key) {
        if (cache.containsKey(key)) {
            System.out.println("Cache Hit!");
            return cache.get(key);
        }
        System.out.println("Cache Miss!");
        String content = provider.getContent(key);
```

```
        cache.put(key, content);
        return content;
    }
}
```

Client.java

```
public class Client {
    public static void main(String[] args) {
        Middleware middleware = new Middleware();
        System.out.println("\nFirst Request");
        System.out.println(middleware.requestContent("Java"));
        System.out.println("\nSecond Request");
        System.out.println(middleware.requestContent("Java"));
        System.out.println("\nThird Request");
        System.out.println(middleware.requestContent("Python"));
        System.out.println("\nFourth Request");
        System.out.println(middleware.requestContent("Python"));
    }
}
```

Run: `javac *.java` then `java Client`

Expected output

```
First Request
Cache Miss!
Fetching from Content Provider...
Java is an Object Oriented Programming Language

Second Request
Cache Hit!
Java is an Object Oriented Programming Language

Third Request
Cache Miss!
Fetching from Content Provider...
Python is a High Level Programming Language

Fourth Request
Cache Hit!
Python is a High Level Programming Language
```

LA4: Berkeley algorithm

BerkeleyAlgorithm.java

```
import java.util.Scanner;

public class BerkeleyAlgorithm {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of clocks: ");
        int n = sc.nextInt();
        int[] clock = new int[n];
        System.out.println("Enter clock times:");
        for (int i = 0; i < n; i++) {
            System.out.print("Clock " + (i + 1) + ": ");
            clock[i] = sc.nextInt();
        }
        int sum = 0;
        for (int i = 0; i < n; i++) {
            sum += clock[i];
        }
        int average = sum / n;
        System.out.println("\nAverage Time = " + average);
        System.out.println("\nAdjustment Values:");
        for (int i = 0; i < n; i++) {
            int adjustment = average - clock[i];
            System.out.println("Clock " + (i + 1) + " Adjustment = " +
adjustment);
        }
        System.out.println("\nSynchronized Clocks:");
        for (int i = 0; i < n; i++) {
            clock[i] = average;
            System.out.println("Clock " + (i + 1) + " = " + clock[i]);
        }
    }
}
```

Run: `javac BerkeleyAlgorithm.java` then `java BerkeleyAlgorithm`

Expected output

```
Enter number of clocks: 4
Enter clock times:
Clock 1: 100
Clock 2: 110
Clock 3: 90
Clock 4: 105

Average Time = 101

Adjustment Values:
Clock 1 Adjustment = 1
Clock 2 Adjustment = -9
Clock 3 Adjustment = 11
Clock 4 Adjustment = -4

Synchronized Clocks:
Clock 1 = 101
Clock 2 = 101
Clock 3 = 101
Clock 4 = 101
```

Formula: `adjustment = average - clock`. The average is (100+110+90+105)/4 = 101.25, which integer division makes 101.

LA5: Token ring mutual exclusion

TokenRing.java

```
import java.util.Scanner;

public class TokenRing {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of processes: ");
        int n = sc.nextInt();
        int token;
        System.out.print("Enter process to initialize token (0 to " + (n - 1)
+ "): ");
        token = sc.nextInt();
        while (true) {
            System.out.println("\nToken is currently with Process P" +
```

```
token);
            System.out.print("Enter process number requesting Critical
Section: ");
            int request = sc.nextInt();
            while (token != request) {
                System.out.println("Token passed from P" + token + " to P" +
((token + 1) % n));
                token = (token + 1) % n;
            }
            System.out.println("\nProcess P" + request + " has entered
Critical Section");
            System.out.println("Process P" + request + " is executing...");
            System.out.println("Process P" + request + " has exited Critical
Section");
            token = (token + 1) % n;
            System.out.println("Token passed to Process P" + token);
            System.out.print("\nDo you want to continue? (1=Yes, 0=No): ");
            int choice = sc.nextInt();
            if (choice == 0)
                break;
        }
        System.out.println("\nSimulation Completed");
    }
}
```

Run: `javac TokenRing.java` then `java TokenRing`

Expected output

```
Enter number of processes: 5
Enter process to initialize token (0 to 4): 0

Token is currently with Process P0
Enter process number requesting Critical Section: 3
Token passed from P0 to P1
Token passed from P1 to P2
Token passed from P2 to P3

Process P3 has entered Critical Section
Process P3 is executing...
Process P3 has exited Critical Section
Token passed to Process P4
```

```
Do you want to continue? (1=Yes, 0=No): 1

Token is currently with Process P4
Enter process number requesting Critical Section: 1
Token passed from P4 to P0
Token passed from P0 to P1

Process P1 has entered Critical Section
Process P1 is executing...
Process P1 has exited Critical Section
Token passed to Process P2

Do you want to continue? (1=Yes, 0=No): 0

Simulation Completed
```

Key line: `token = (token + 1) % n;` passes the token to the next process, and the `%` makes the ring wrap round to P0.

(LA6 to LA10 continue below.)

LA6: Bully and Ring election

Your zip has the two algorithms as separate files. Run both with the same input: 5 processes, failed process 5, initiator 2. (Your LA 6 manual also has a combined `ElectionComparison.java` that runs both in one go.)

BullyElection.java

```
import java.util.Scanner;

public class BullyElection {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of processes: ");
        int n = sc.nextInt();
        int coordinator = n;
        System.out.println("Current Coordinator: P" + coordinator);
        System.out.print("Enter failed coordinator process: ");
        int failed = sc.nextInt();
        System.out.print("Enter process initiating election: ");
        int initiator = sc.nextInt();
```

```
        System.out.println("\nElection Process:");
        for (int i = initiator + 1; i <= n; i++) {
            if (i != failed) {
                System.out.println("P" + initiator + " sends ELECTION to P" +
i);
            }
        }
        int newCoordinator = 0;
        for (int i = n; i >= 1; i--) {
            if (i != failed) {
                newCoordinator = i;
                break;
            }
        }
        System.out.println("\nNew Coordinator = P" + newCoordinator);
        System.out.println("Coordinator Announcement:");
        for (int i = 1; i <= n; i++) {
            if (i != newCoordinator && i != failed) {
                System.out.println("P" + newCoordinator + " informs P" + i);
            }
        }
    }
}
```

RingElection.java

```
import java.util.Scanner;

public class RingElection {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of processes: ");
        int n = sc.nextInt();
        boolean[] active = new boolean[n + 1];
        for (int i = 1; i <= n; i++)
            active[i] = true;
        System.out.print("Enter failed process: ");
        int failed = sc.nextInt();
        active[failed] = false;
        System.out.print("Enter initiator process: ");
        int initiator = sc.nextInt();
        System.out.println("\nElection Message Passing:");
        int max = initiator;
```

```
        int current = initiator;
        do {
            int next = current % n + 1;
            while (!active[next])
                next = next % n + 1;
            System.out.println("P" + current + " -> P" + next);
            if (next > max)
                max = next;
            current = next;
        } while (current != initiator);
        System.out.println("\nNew Coordinator = P" + max);
        System.out.println("\nCoordinator Announcement:");
        current = max;
        do {
            int next = current % n + 1;
            while (!active[next])
                next = next % n + 1;
            System.out.println("Coordinator Message: P" + current + " -> P" +
next);
            current = next;
        } while (current != max);
    }
}
```

Run: `javac *.java`, then `java BullyElection` and `java RingElection`

Expected output, Bully

```
Enter number of processes: 5
Current Coordinator: P5
Enter failed coordinator process: 5
Enter process initiating election: 2

Election Process:
P2 sends ELECTION to P3
P2 sends ELECTION to P4

New Coordinator = P4
Coordinator Announcement:
P4 informs P1
P4 informs P2
P4 informs P3
```

Expected output, Ring

```
Enter number of processes: 5
Enter failed process: 5
Enter initiator process: 2

Election Message Passing:
P2 -> P3
P3 -> P4
P4 -> P1
P1 -> P2

New Coordinator = P4

Coordinator Announcement:
Coordinator Message: P4 -> P1
Coordinator Message: P1 -> P2
Coordinator Message: P2 -> P3
Coordinator Message: P3 -> P4
```

Compare: Bully sends ELECTION only to higher IDs and announces to everyone. Ring passes one message round the ring (skipping the failed P5). Both choose P4.

LA7: Distributed caching with replication

CacheNode.java

```
import java.util.HashMap;

public class CacheNode {
    private String nodeName;
    private HashMap<String, String> cache;

    public CacheNode(String nodeName) {
        this.nodeName = nodeName;
        cache = new HashMap<>();
    }
    public void put(String key, String value) {
        cache.put(key, value);
        System.out.println(nodeName + " stored : " + key + " -> " + value);
    }
    public String get(String key) {
```

```
        return cache.get(key);
    }
    public boolean contains(String key) {
        return cache.containsKey(key);
    }
    public String getNodeName() {
        return nodeName;
    }
}
```

DistributedCache.java

```
public class DistributedCache {
    private CacheNode node1;
    private CacheNode node2;

    public DistributedCache() {
        node1 = new CacheNode("Node1");
        node2 = new CacheNode("Node2");
    }
    public void put(String key, String value) {
        System.out.println("\nStoring data in distributed cache");
        node1.put(key, value);
        System.out.println("Replicating to Node2");
        node2.put(key, value);
    }
    public String get(String key) {
        if (node1.contains(key)) {
            System.out.println("Found in Node1");
            return node1.get(key);
        }
        if (node2.contains(key)) {
            System.out.println("Found in Node2");
            return node2.get(key);
        }
        return "Data Not Found";
    }
}
```

Main.java

```
public class Main {
    public static void main(String[] args) {
        DistributedCache cache = new DistributedCache();
        cache.put("Java", "Object Oriented Programming Language");
        cache.put("AI", "Artificial Intelligence");
        System.out.println("\nSearching Java");
        System.out.println(cache.get("Java"));
        System.out.println("\nSearching AI");
        System.out.println(cache.get("AI"));
        System.out.println("\nSearching Python");
        System.out.println(cache.get("Python"));
    }
}
```

Run: `javac *.java` then `java Main`

Expected output

```
Storing data in distributed cache
Node1 stored : Java -> Object Oriented Programming Language
Replicating to Node2
Node2 stored : Java -> Object Oriented Programming Language

Storing data in distributed cache
Node1 stored : AI -> Artificial Intelligence
Replicating to Node2
Node2 stored : AI -> Artificial Intelligence

Searching Java
Found in Node1
Object Oriented Programming Language

Searching AI
Found in Node1
Artificial Intelligence

Searching Python
Data Not Found
```

LA8: Client-server checkpointing and recovery

CheckpointServer.java

```
import java.io.*;
import java.net.*;
import java.util.*;

public class CheckpointServer {
    private static int counter = 0;

    public static void saveCheckpoint() {
        try {
            FileWriter fw = new FileWriter("checkpoint.txt");
            fw.write(String.valueOf(counter));
            fw.close();
            System.out.println("Checkpoint Saved: " + counter);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    public static void restoreCheckpoint() {
        try {
            BufferedReader br = new BufferedReader(new
FileReader("checkpoint.txt"));
            counter = Integer.parseInt(br.readLine());
            br.close();
            System.out.println("Recovered from Checkpoint: " + counter);
        } catch (Exception e) {
            System.out.println("No checkpoint found.");
        }
    }

    public static void main(String args[]) {
        try {
            ServerSocket server = new ServerSocket(5000);
            System.out.println("Server Started...");
            restoreCheckpoint();
            while (true) {
                Socket client = server.accept();
                counter++;
                saveCheckpoint();
```

```
                DataOutputStream dos = new
DataOutputStream(client.getOutputStream());
                dos.writeUTF("Server Counter = " + counter);
                client.close();
                if (counter == 5) {
                    System.out.println("\n*** Simulated Failure ***");
                    throw new Exception("Server Crash");
                }
            }
        } catch (Exception e) {
            System.out.println("Failure Detected!");
            restoreCheckpoint();
            System.out.println("Server Restored Successfully");
        }
    }
}
```

CheckpointClient.java

```
import java.io.*;
import java.net.*;

public class CheckpointClient {
    public static void main(String args[]) {
        try {
            Socket socket = new Socket("localhost", 5000);
            DataInputStream dis = new
DataInputStream(socket.getInputStream());
            String msg = dis.readUTF();
            System.out.println("Response: " + msg);
            socket.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Run: `javac *.java`. Terminal 1: `java CheckpointServer`. Terminal 2: run `java CheckpointClient` five times. Delete an old `checkpoint.txt` first, so the first run starts from 0.

Expected output, server (Terminal 1)

```
Server Started...
No checkpoint found.
Checkpoint Saved: 1
Checkpoint Saved: 2
Checkpoint Saved: 3
Checkpoint Saved: 4
Checkpoint Saved: 5

*** Simulated Failure ***
Failure Detected!
Recovered from Checkpoint: 5
Server Restored Successfully
```

Expected output, client (Terminal 2, one line per run)

```
Response: Server Counter = 1
Response: Server Counter = 2
Response: Server Counter = 3
Response: Server Counter = 4
Response: Server Counter = 5
```

File check: `cat checkpoint.txt` shows `5`.

LA9: Video recommendation

The only change from your file is the prompt text, where `ci-Fi` is corrected to `Sci-Fi`.

Video.java

```
public class Video {
    private String title;
    private String genre;
    private String language;
    private double rating;

    public Video(String title, String genre, String language, double rating)
{
        this.title = title;
        this.genre = genre;
        this.language = language;
        this.rating = rating;
    }
```

```
    public String getTitle() { return title; }
    public String getGenre() { return genre; }
    public String getLanguage() { return language; }
    public double getRating() { return rating; }

    public void display() {
        System.out.println("Title    : " + title);
        System.out.println("Genre    : " + genre);
        System.out.println("Language : " + language);
        System.out.println("Rating   : " + rating);
        System.out.println("------------------------");
    }
}
```

VideoRecommendation.java

```
import java.util.ArrayList;
import java.util.Scanner;

public class VideoRecommendation {
    public static void main(String[] args) {
        ArrayList<Video> videos = new ArrayList<>();
        videos.add(new Video("Bahubali", "Action", "Hindi", 8.5));
        videos.add(new Video("3 Idiots", "Comedy", "Hindi", 9.0));
        videos.add(new Video("Interstellar", "Sci-Fi", "English", 9.2));
        videos.add(new Video("Avengers Endgame", "Action", "English", 8.9));
        videos.add(new Video("Drishyam", "Thriller", "Hindi", 8.8));

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Preferred Genre (Action, Comedy, Sci-Fi,
Thriller) : ");
        String userGenre = sc.nextLine();
        System.out.print("Enter Preferred Language : (Hindi,  English) ");
        String userLanguage = sc.nextLine();

        System.out.println("\nRecommended Videos\n");
        boolean found = false;
        for (Video v : videos) {
            if (v.getGenre().equalsIgnoreCase(userGenre)
                    && v.getLanguage().equalsIgnoreCase(userLanguage)) {
                v.display();
                found = true;
            }
```

```
        }
        if (!found) {
            System.out.println("No matching videos found.");
        }
        sc.close();
    }
}
```

Run: `javac *.java` then `java VideoRecommendation`

Expected output, run 1

```
Enter Preferred Genre (Action, Comedy, Sci-Fi, Thriller) : Action
Enter Preferred Language : (Hindi,  English) Hindi

Recommended Videos

Title    : Bahubali
Genre    : Action
Language : Hindi
Rating   : 8.5
------------------------
```

Expected output, run 2

```
Enter Preferred Genre (Action, Comedy, Sci-Fi, Thriller) : Horror
Enter Preferred Language : (Hindi,  English) Hindi

Recommended Videos

No matching videos found.
```

LA10: Web cluster with load balancer and proxy cache

WebCluster.java (the WebServer class is in the same file)

```
import java.util.*;

class WebServer {
    int id;

    WebServer(int id) {
```

```
        this.id = id;
    }
    public String processRequest(String request) {
        return "Response from Server-" + id + " for Request: " + request;
    }
}

public class WebCluster {
    static HashMap<String, String> cache = new HashMap<>();

    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        WebServer[] servers = { new WebServer(1), new WebServer(2), new
WebServer(3) };
        int currentServer = 0;
        while (true) {
            System.out.print("\nEnter Request (or exit): ");
            String request = sc.nextLine();
            if (request.equalsIgnoreCase("exit"))
                break;
            if (cache.containsKey(request)) {
                System.out.println("\nCACHE HIT");
                System.out.println(cache.get(request));
                continue;
            }
            System.out.println("\nCACHE MISS");
            WebServer server = servers[currentServer];
            currentServer = (currentServer + 1) % servers.length;
            String response = server.processRequest(request);
            cache.put(request, response);
            System.out.println("Load Balancer Routed To Server-" +
server.id);
            System.out.println(response);
        }
        System.out.println("\nSystem Shutdown");
    }
}
```

Run: `javac WebCluster.java` then `java WebCluster`

Expected output

```
Enter Request (or exit): home

CACHE MISS
Load Balancer Routed To Server-1
Response from Server-1 for Request: home

Enter Request (or exit): about

CACHE MISS
Load Balancer Routed To Server-2
Response from Server-2 for Request: about

Enter Request (or exit): contact

CACHE MISS
Load Balancer Routed To Server-3
Response from Server-3 for Request: contact

Enter Request (or exit): home

CACHE HIT
Response from Server-1 for Request: home

Enter Request (or exit): exit

System Shutdown
```

Key line: `currentServer = (currentServer + 1) % servers.length;` is round robin. A repeated request (`home`) is answered from the cache with no server involved.

Quick revision

- Always compile with `javac *.java` in the folder that holds the files, then run `java ClassName`.
- RMI: start `java Server` first, then the clients. CORBA: tnameserv, Server, Client, each with `-ORBInitialPort 1050`.
- Checkpoint: start the server first, and delete `checkpoint.txt` to restart from zero.
- Interactive programs (LA4, LA5, LA6, LA9, LA10) read from the keyboard, so type the values shown after each prompt.